

# UltimateDashboard

## Documento maestro de producto, diseño y desarrollo

Versión 1.0 · 8 de octubre de 2026 · Nombre provisional

**Versión:** 1.0 · **Estado:** baseline de planificación · **Fecha:** 8 octubre 2026 · **Nombre:** provisional

Fuente de verdad del proyecto. Una decisión aprobada se registra como ADR. Las propuestas no se convierten en requisitos sin aprobación.

### 1. Visión y principios

UltimateDashboard es una suite web modular para organización personal y colaboración en espacios compartidos, con estética minimalista, cálida y personalizable. Se desplegará en Vercel con Next.js, TypeScript y Supabase; se desarrollará con Codex en VS Code.

**Principios:** privacidad por defecto; una fuente de verdad por dato; simplicidad operativa; accesibilidad; rendimiento medible; automatización reversible; cambios pequeños; pruebas antes de despliegue; evitar sobreingeniería.

**Fuera de alcance:** gestión financiera, nóminas, extractos bancarios y facturas; Apple Health; microservicios; calendario nativo duplicado de Google; publicación comercial y pagos.

### 2. Decisiones aprobadas y pendientes

**Aprobado:** diseño minimalista, acogedor y personalizable; multiusuario; espacios privados y compartidos; módulos activables por espacio; roles administrador/editor/lector; tareas híbridas; notas avanzadas tipo Notion; calendario Google bidireccional; tareas visibles en calendario sin convertirse automáticamente en eventos; salud manual; catálogo IGDB/TMDB sujeto a condiciones; doce módulos; privacidad de salud; nombre provisional UltimateDashboard.

**Propuesta técnica a validar:** Next.js App Router, Tailwind CSS, shadcn/ui, Tiptap, TanStack Query donde aporte valor, Zod, React Hook Form, Vitest, Playwright, Sentry u observabilidad equivalente. Versiones exactas se fijarán tras comprobar compatibilidad al crear el repositorio.

**Pendiente:** disponibilidad de modelos en Codex, credenciales y cuotas de APIs externas, diseño de recordatorios, política de retención, estrategia de colaboración simultánea en notas, idiomas y zona horaria por usuario, catálogo de plantillas, importación/exportación y límites de adjuntos.

### 3. Módulos y requisitos

#### 3.1 Dashboard personalizable

Widgets añadibles/eliminables, reordenación, tamaños, ajustes por widget, temas y modo oscuro, múltiples vistas y persistencia por usuario. Puede mostrar información autorizada de varios espacios. Debe funcionar en móvil con reflujo y accesibilidad por teclado.

#### 3.2 Google Calendar

OAuth por usuario; listado de calendarios accesibles; vistas día/semana/mes/agenda; filtros; crear, editar y borrar eventos con confirmación apropiada y permisos de Google. Los eventos de Google siguen siendo propiedad de Google. Las tareas de Supabase se superponen visualmente sin escritura automática en Google. Gestionar revocación de permisos,

renovación de tokens, zonas horarias, recurrencia, fallos y sincronización incremental cuando corresponda.

### 3.3 Tareas

Captura rápida, bandeja de entrada, listas, proyectos, subtareas, prioridades, fechas, recurrencia, etiquetas, asignados, Kanban, filtros y búsqueda. "Mis tareas" agrega tareas privadas y compartidas asignadas. La pertenencia al espacio gobierna visibilidad y permisos.

### 3.4 Notas

Páginas jerárquicas y editor por bloques (Tiptap candidato), texto enriquecido, tablas, imágenes, código, enlaces internos, plantillas, adjuntos, búsqueda y versiones. Colaboración concurrente en tiempo real es una fase separada: no asumir que Supabase Realtime resuelve edición CRDT por sí solo.

### 3.5 Peso y medidas

Registro manual de peso, medidas, objetivos, evolución y gráficos. Datos exclusivamente privados; evitar incluirlos en búsquedas y espacios compartidos. Sin diagnóstico ni recomendaciones clínicas automáticas.

### 3.6 Gimnasio y bonos

Sesiones, ejercicios, duración, notas, estado; bonos con sesiones compradas, utilizadas, restantes, caducidad y coste unitario opcional. Consumir una sesión únicamente al marcarla completada; cancelación no descuenta. Evitar doble descuento mediante transacción idempotente.

### 3.7 Entretenimiento

Videojuegos: wishlist, jugando, completado, abandonado, horas, valoración. Películas: pendiente, vistas, favoritas, valoración. Series: seguimiento por temporadas y episodios. Búsqueda de metadatos/caratulas mediante IGDB y TMDb tras verificar licencias, cuotas, atribución y autenticación. Datos de seguimiento propios en Supabase; caché controlada de metadatos.

### 3.8 Listas de la compra

Listas personales o compartidas, artículos frecuentes, cantidades, unidades, categorías, marcado en tiempo real y reutilización. Preparadas para importar ingredientes desde menús; evitar duplicados por normalización de unidad y producto.

### 3.9 Hábitos y rutinas

Hábitos diarios/semanales, objetivos, check-ins, rachas, progreso, historial y recordatorios opcionales. Zona horaria y reglas de racha explícitas; no penalizar automáticamente por días sin datos sin configuración.

### 3.10 Suscripciones y vencimientos

Registro de servicio, periodicidad, próxima renovación/caducidad, recordatorios y estado. Solo gestión de fechas y avisos: **no** análisis financiero ni conexión bancaria. Los importes, si se añaden, serán metadatos opcionales sin módulo de finanzas.

### 3.11 Proyectos personales

Proyectos, etapas, hitos, progreso, tareas relacionadas, notas y archivos, con vista resumen y fechas. La relación con tareas usa referencias, no copias.

### 3.12 Recetas y menús

Recetas, ingredientes, cantidades, pasos, duración, etiquetas dietéticas y alérgenos; planificación semanal o quincenal; generación revisable de listas de compra. Las restricciones alimentarias se configuran por hogar/persona y nunca se infieren automáticamente. Diferenciar ingrediente, unidad y cantidad; permitir ajustes de raciones.

## 4. Espacios, identidad y autorización

Cada usuario tiene espacio personal intransferible. Los espacios compartidos incluyen membresías y roles `admin`, `editor`, `viewer`; activación de módulos por espacio. Cada entidad compartible lleva `space_id` y las políticas de acceso se comprueban en PostgreSQL mediante RLS, no solo en la UI. Salud solo admite `owner_user_id` y no `space_id` compartido. Un dashboard puede combinar widgets de espacios distintos sin otorgar acceso adicional.

**Matriz base:** admin administra espacio/miembros/módulos y contenido; editor crea y modifica contenido; viewer lee. Eliminación de espacio, expulsión y transferencia requieren flujos explícitos. Definir propiedad y tratamiento de datos tras salida de miembros antes de implementar invitaciones.

## 5. Arquitectura de referencia

- **Frontend y BFF:** Next.js App Router, React, TypeScript estricto; Server Components por defecto y Client Components para interacción; rutas/acciones de servidor validadas.
- **UI:** Tailwind, tokens semánticos, componentes accesibles; shadcn/ui como base editable, sin acoplar lógica de negocio a la vista.
- **Persistencia:** Supabase Postgres, migraciones SQL versionadas, RLS, Auth, Storage privado y Realtime solo donde sea necesario.
- **Integraciones:** adaptadores de Google Calendar, TMDb e IGDB en servidor; secretos fuera del cliente; controles de cuota, caché, reintentos limitados y auditoría.
- **Notificaciones:** arquitectura desacoplada; cron/colas solo cuando se definan garantías de entrega y proveedor.
- **Despliegue:** Vercel preview por PR, staging y producción; Supabase separado por entorno cuando sea viable.

**Flujo:** UI → casos de uso del módulo → repositorios/adaptadores → Supabase o proveedor externo. Evitar que componentes React ejecuten consultas privilegiadas o dependan de SDKs de proveedores directamente.

## 6. Modelo de datos inicial (conceptual)

profiles, spaces, space\_members, space\_modules, dashboard\_layouts, dashboard\_widgets, tasks, task\_assignments, task\_labels, projects, project\_milestones, notes, note\_versions, attachments, google\_connections, google\_calendar\_preferences, weight\_entries, body\_measurements, workouts, gym\_packages, gym\_package\_consumptions, media\_items, media\_progress, shopping\_lists, shopping\_items, habits, habit\_checkins, renewals, recipes, recipe\_ingredients, meal\_plans, meal\_plan\_entries, notifications, audit\_events.

Convenciones: PK UUID; `created_at`, `updated_at`; claves externas explícitas; índices en `space_id`, `owner_user_id`, fechas y asignaciones; restricciones únicas de idempotencia; borrado lógico solo donde aporte valor; migraciones hacia delante y estrategia de rollback. No crear todas las tablas al inicio: migrar por vertical funcional.

## 7. Sistema de diseño

**Dirección:** minimalismo cálido y personal, no dashboard empresarial genérico. Fondos neutros cálidos, superficies suaves, acentos moderados, tipografía legible, radios consistentes, iconografía sencilla y abundante espacio negativo. Tokens semánticos (`surface`, `text`, `muted`, `accent`, `border`, `danger`, `success`) para temas personalizables y dark mode. No codificar colores por módulo directamente en componentes.

**UX:** navegación lateral en escritorio, navegación adaptada a móvil, búsqueda global, captura rápida, estados vacíos útiles, skeletons, errores recuperables, confirmación para acciones destructivas, teclado, foco visible y objetivos WCAG 2.2 AA. PWA instalable con estrategia offline explícita y limitada; no prometer sincronización offline completa en MVP.

**Dashboard:** rejilla configurable con límites de tamaño y prioridades de móvil; los widgets no deben acceder a datos sin autorización. Guardar layout y preferencias por usuario, no por navegador.

## 8. Seguridad y privacidad

RLS obligatoria en tablas con datos de usuario; pruebas positivas y negativas para acceso cruzado. No usar service role en cliente. Secretos de Google/IGDB/TMDB solo servidor; cifrado y gestión segura de tokens OAuth, rotación y revocación. Storage privado con políticas y URLs firmadas de corta duración. Validación de entrada y salida, límites de tamaño, sanitización del editor, CSP y protección frente a XSS/CSRF según mecanismo de sesión. Auditoría de cambios sensibles. Política de backup, recuperación y borrado/exportación de datos antes de producción.

**Regla de publicación:** ninguna función compartida se da por terminada sin prueba de aislamiento entre dos usuarios y dos espacios.

## 9. Agentes Codex y modelos

**Orquestador / Tech Lead:** divide entregas, selecciona agente, vigila dependencias, revisa criterios. Modelo: mejor modelo de codificación disponible en Codex, razonamiento alto en decisiones complejas.

**Arquitectura:** límites de módulos, ADR, contratos y diseño de datos. Modelo de codificación más capaz disponible, razonamiento alto.

**Frontend / Design System:** componentes, responsive, accesibilidad y pruebas de interfaz. Modelo de codificación disponible, razonamiento medio; alto para refactors transversales.

**Backend / Integraciones:** casos de uso, APIs, OAuth, caché y errores. Modelo de codificación disponible, razonamiento alto en OAuth y sincronización.

**DB / Seguridad:** migraciones, RLS, índices, aislamiento y revisiones de privilegios. Modelo de codificación más capaz disponible, razonamiento alto.

**QA:** estrategia de pruebas, fixtures, unitarias/integración/E2E, revisión de regresiones. Modelo de codificación disponible, razonamiento medio-alto.

**Documentación / Release:** README, ADR, changelog, checklist de despliegue. Modelo rápido disponible, razonamiento bajo-medio; revisión humana para contenido normativo.

**Política de selección de modelo:** no fijar identificadores ni precios no verificados. Al inicializar Codex, consultar modelos y niveles de razonamiento realmente disponibles; registrar en [docs/architecture/ai-model-policy.md](#) el identificador exacto, tareas permitidas, coste observado y fecha. Escalar a modelo más capaz solo por riesgo, complejidad o fallo de una solución anterior. Un agente no equivale necesariamente a un proceso simultáneo: comenzar con orquestación secuencial y delegación selectiva.

**Contrato de cada tarea para agente:** objetivo, contexto, archivos permitidos, dependencias, criterios de aceptación, pruebas obligatorias, restricciones, resultado esperado y riesgos. Cada agente devuelve archivos tocados, decisiones, pruebas ejecutadas y asuntos abiertos.

## 10. Flujo de desarrollo y calidad

1. Abrir issue con historia, alcance, no-alcance y criterios de aceptación.
2. Elaborar plan breve para cambios medianos; ADR para decisiones arquitectónicas.
3. Implementar vertical pequeño con validación de tipos y pruebas.
4. Ejecutar lint, format, typecheck, unitarias e integración; Playwright para flujos críticos.
5. Revisar RLS y permisos si toca datos compartidos; revisar accesibilidad si toca UI.
6. PR con resumen, capturas cuando proceda, migraciones, riesgos y plan de reversión.
7. Preview en Vercel; validación manual; aprobación humana para producción.
8. Actualizar documentación y registro de decisiones.

**Definition of Done:** criterios cumplidos, sin secretos, TypeScript estricto, pruebas verdes, errores manejados, accesibilidad comprobada, telemetría suficiente, migración segura, docs actualizadas y revisión aprobada.

**CI recomendado:** lint → typecheck → test:unit → test:integration (DB temporal) → build → test:e2e en preview para rutas críticas → comprobaciones de dependencias/secretos. Establecer umbrales de cobertura por código crítico, no un porcentaje global arbitrario.

## 11. Roadmap por entregas

**Fase 0 — Foundation:** repositorio, AGENTS.md, normas, CI, entornos, tokens UI, autenticación, perfiles, modelo mínimo de espacios, RLS y pruebas de aislamiento.

**Fase 1 — Núcleo utilizable:** dashboard básico personalizable, tareas, espacios compartidos, búsqueda inicial, diseño responsive y PWA mínima.

**Fase 2 — Productividad:** Google Calendar OAuth/sincronización, notas básicas y proyectos; después editor avanzado/versiones.

**Fase 3 — Hogar:** compras, recetas, menús y generación de lista; renovaciones.

**Fase 4 — Bienestar:** peso, medidas, entrenos, bonos, hábitos.

**Fase 5 — Entretenimiento:** catálogo externo, biblioteca y progreso de series.

**Fase 6 — Madurez:** notificaciones, importación/exportación, rendimiento, auditoría, colaboración avanzada en notas si se aprueba.

Cada fase tendrá demo funcional, criterios de salida, migraciones, tests y deuda técnica registrada. No iniciar varias integraciones externas en paralelo antes de estabilizar autenticación y permisos.

## 12. Primera entrega recomendada (tickets)

- F0-01 Crear repo Next.js con TypeScript estricto, lint, format y scripts de prueba.
- F0-02 Definir tokens, layout shell y componentes de navegación responsive.
- F0-03 Configurar Supabase Auth y perfiles con rutas protegidas.
- F0-04 Crear espacios personales y membresías compartidas, migración + RLS.
- F0-05 Añadir pruebas de acceso cruzado y pipeline CI.
- F0-06 Crear dashboard vacío configurable y persistencia de widgets.
- F0-07 Crear plantilla de PR, ADR y checklist de seguridad.

## 13. Métricas y presupuestos de rendimiento

Medir Core Web Vitals reales (LCP, INP, CLS), latencia de operaciones críticas, errores de integración, tasa de éxito de sincronización, tamaño del bundle y consultas lentas. Definir objetivos cuantitativos tras instrumentar el primer vertical; no prometer cifras sin línea base. Usar paginación, índices, caché y carga diferida antes de añadir infraestructura.

## 14. Gestión del cambio

Las decisiones se documentan como ADR-0001...; todo cambio de alcance requiere impacto en datos, permisos, UX, pruebas y cronograma. MASTER\_SPEC.md describe qué construir; AGENTS.md describe cómo debe trabajar Codex; el código y las migraciones son la implementación efectiva. Si hay contradicción, detenerse y pedir decisión, no inventar una regla.